```
rooms = {'A': 1, 'B': 1}
position = 'A'

while 0 in rooms.values():
    print("Position:", position)

    if rooms[position] == 0:
        print("Cleaning room", position)
        rooms[position] = 1
    else:
        print("Room is clean")

    position = 'B' if position == 'A' else 'A'

print("Both rooms are clean!")
```

```
Both rooms are clean!
```

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(n for n in graph[node] if n not in visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

bfs(graph, 'A')
```

```
A B C D E F
```

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(graph, neighbour, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

dfs(graph, 'A', set())
```

```
A B D E C F
```

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
```

```
]

n = len(graph)
min_cost = float('inf')
best_path = None

for path in permutations(range(1, n)):
    route = (0,) + path + (0,)
    cost = sum(graph[route[i]][route[i+1]] for i in range(n))

    if cost < min_cost:
        min_cost = cost
        best_path = route

print("Best Path:", best_path)
print("Minimum Cost:", min_cost)
```

```
Best Path: (0, 1, 3, 2, 0)
Minimum Cost: 80
```

```
def astar(graph, h, start, goal):
    open_list = [(h[start], start, 0)]
    visited = set()

    while open_list:
        open_list.sort()
        f, node, g = open_list.pop(0)

        if node == goal:
            print("Goal reached:", node)
            print("Cost:", g)
            return

        if node not in visited:
            visited.add(node)

            for neighbour, cost in graph[node]:
                if neighbour not in visited:
                    new_g = g + cost
                    new_f = new_g + h[neighbour]
                    open_list.append((new_f, neighbour, new_g))

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

h = {'A': 5, 'B': 4, 'C': 4, 'D': 2, 'E': 1, 'F': 2, 'G': 0}

astar(graph, h, 'A', 'G')
```

```
Goal reached: G
Cost: 4
```

```
colors = ['Red', 'Green', 'Blue']

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C'],
    'C': ['A', 'B']
}

coloring = {}

def csp(node):
    if node == len(graph):
        return True

    state = list(graph.keys())[node]

    for color in colors:
        if all(coloring.get(n) != color for n in graph[state]):
            coloring[state] = color
```

```
            if csp(node + 1):
                return True

            del coloring[state]

    return False

if csp(0):
    print(coloring)
else:
    print("No solution")
```

```
{'A': 'Red', 'B': 'Green', 'C': 'Blue'}
```

```
board = [' '] * 9

def display():
    for i in range(0, 9, 3):
        print('|'.join(board[i:i+3]))
        if i < 6:
            print('-+-+-')

player = 'X'

for turn in range(9):
    display()
    pos = int(input(f"Player {player}, enter position (1-9): ")) - 1

    if pos < 0 or pos > 8 or board[pos] != ' ':
        print("Invalid move!")
        continue

    board[pos] = player

    wins = [(0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)]

    if any(all(board[i] == player for i in line) for line in wins):
        display()
        print("Player", player, "wins!")
        break

    player = 'O' if player == 'X' else 'X'
else:
    display()
    print("It's a draw!")
```

```
 | |
-+-+-
 | |
-+-+-
 | |
Player X, enter position (1-9): 2
 |X|
-+-+-
 | |
-+-+-
 | |
Player O, enter position (1-9): 4
 |X|
-+-+-
O| |
-+-+-
 | |
Player X, enter position (1-9): 1
X|X|
-+-+-
O| |
-+-+-
 | |
Player O, enter position (1-9): 7
X|X|
-+-+-
O| |
-+-+-
O| |
Player X, enter position (1-9): 3
X|X|X
-+-+-
O| |
```

```
-+-+-
O| |
Player X wins!
```

```
def minimax(board, is_max):
    wins = [(0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)]

    for a, b, c in wins:
        if board[a] == board[b] == board[c] != ' ':
            return 1 if board[a] == 'X' else -1

    if ' ' not in board:
        return 0

    scores = []
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X' if is_max else 'O'
            scores.append(minimax(board, not is_max))
            board[i] = ' '

    return max(scores) if is_max else min(scores)

board = [' '] * 9
player = 'X'

for turn in range(9):
    if player == 'X':
        best, move = -float('inf'), -1
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(board, False)
                board[i] = ' '
                if score > best:
                    best, move = score, i
        board[move] = 'X'
    else:
        move = int(input("Enter position (1-9): ")) - 1
        if move < 0 or move > 8 or board[move] != ' ':
            print("Invalid move!")
            continue
        board[move] = 'O'

    for i in range(0, 9, 3):
        print('|'.join(board[i:i+3]))
        print("-----")

    if minimax(board, True) == 1 or minimax(board, False) == -1:
        print("Game over!")
        break

    player = 'O' if player == 'X' else 'X'
```

```
X| |
-----
 | |
-----
 | |
-----
Game over!
```

```
def alphabeta(board, alpha, beta, is_max):
    wins = [(0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)]

    for a, b, c in wins:
        if board[a] == board[b] == board[c] != ' ':
            return 1 if board[a] == 'X' else -1

    if ' ' not in board:
        return 0

    if is_max:
        best = -float('inf')
        for i in range(9):
            if board[i] == ' ':
```

```
                board[i] = 'X'
                best = max(best, alphabeta(board, alpha, beta, False))
                board[i] = ' '
                alpha = max(alpha, best)
                if beta <= alpha:
                    break
        return best
    else:
        best = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                best = min(best, alphabeta(board, alpha, beta, True))
                board[i] = ' '
                beta = min(beta, best)
                if beta <= alpha:
                    break
        return best

board = [' '] * 9

best_score = -float('inf')
best_move = -1

for i in range(9):
    board[i] = 'X'
    score = alphabeta(board, -float('inf'), float('inf'), False)
    board[i] = ' '

    if score > best_score:
        best_score = score
        best_move = i

print("Best move for X:", best_move + 1)
print("Score:", best_score)
```

```
Best move for X: 1
Score: 0
```

```
from sklearn.tree import DecisionTreeClassifier

X = [[1, 1], [1, 0], [0, 1], [0, 0]]
y = [1, 1, 0, 0]

model = DecisionTreeClassifier()
model.fit(X, y)

prediction = model.predict([[1, 0]])
print("Prediction:", prediction[0])
```

```
Prediction: 1
```

```
Start coding or generate with AI.
```